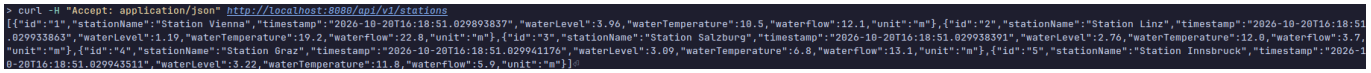
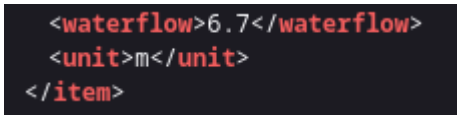
Task 1

Starten und überprüfen

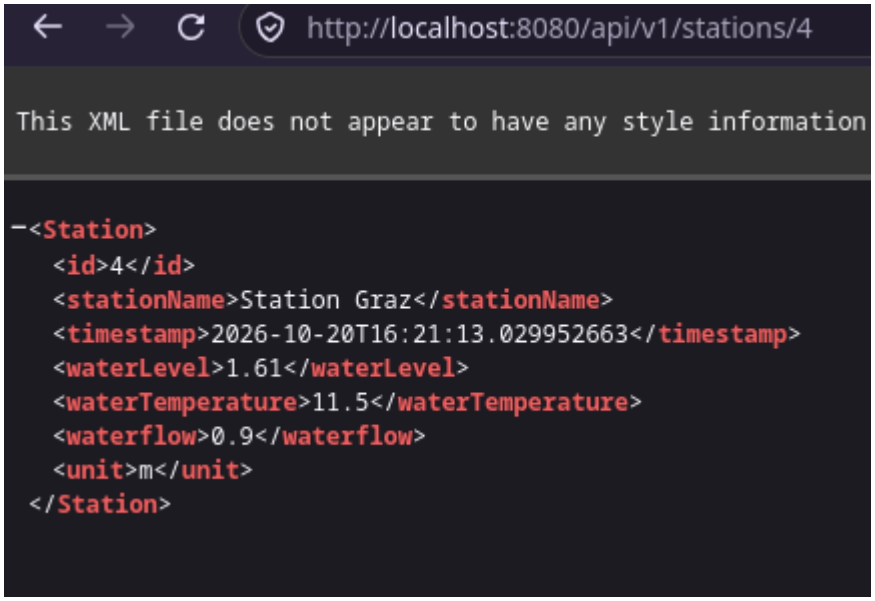


Waterflow hinzufügen



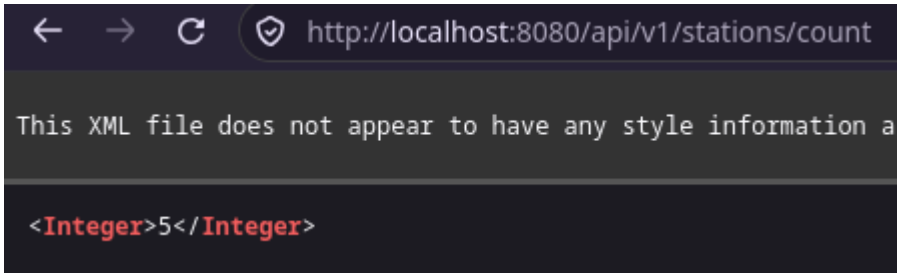
(xml response von später)

Filter hinzufügen für eine einzelne station



Task 2

Count Hinzufügen



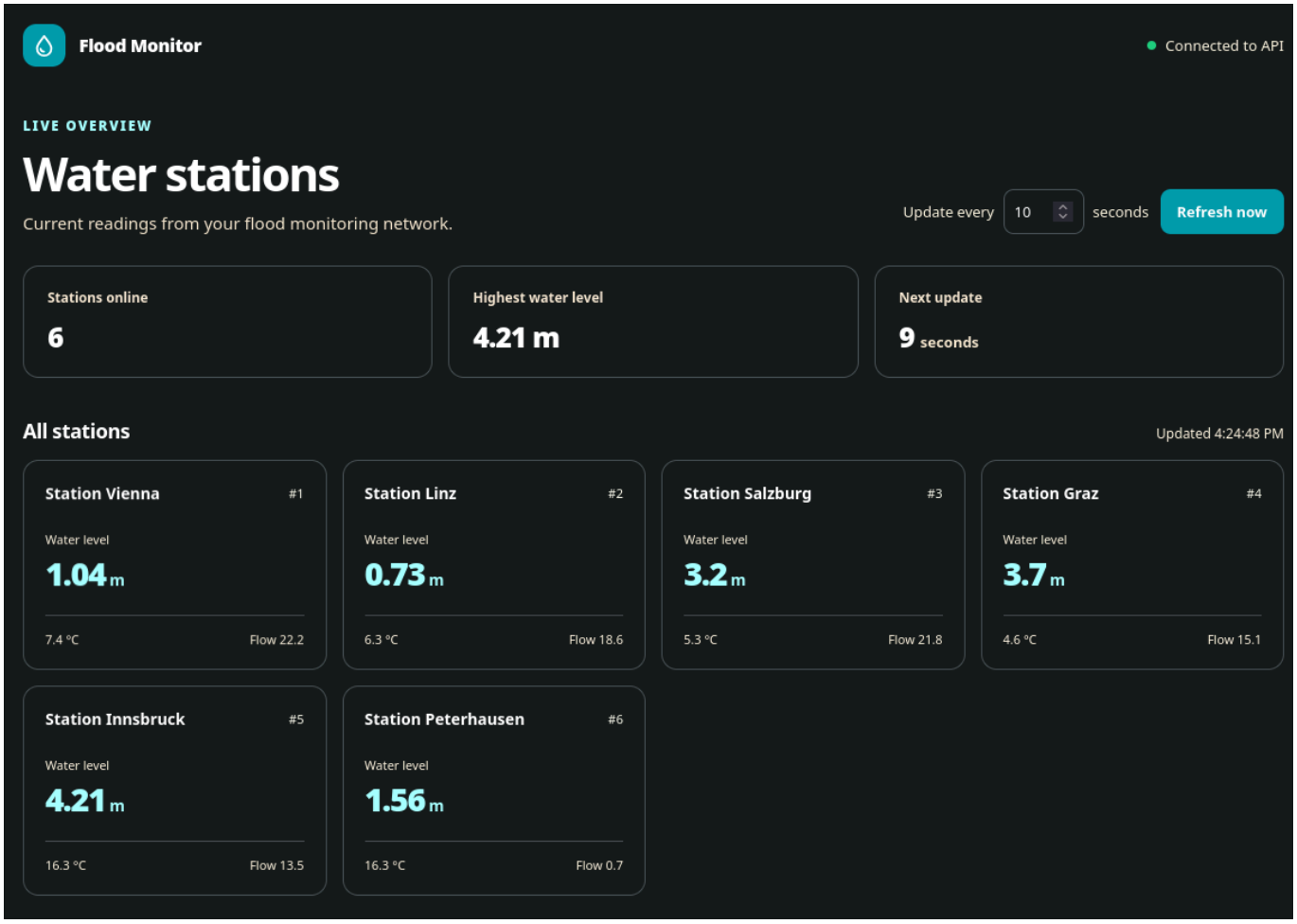
404 Error code

Das macht Spring für uns automatisch

POST hinzufügen

```
curl -i -X POST http://localhost:8080/api/v1/stations \
        -H "Content-Type: application/json" \
        -H "Accept: application/json" \
        -d '{
      "stationName": "Station Peterhausen",
      "timestamp": "2026-10-20T14:30:00",
      "waterLevel": 2.45,
      "waterTemperature": 16.8,
      "waterflow": 12.3,
      "unit": "m"
    }'
```

Hiermit prüfen wir ob das POST interface funktioniert.



Eine Station Peterhausen erscheint dazu.

Task 3

XML support

Spring boot macht automatisch XML support.

Man muss nichts ändern um die daten in XML zu bekommen



Task 4

Fragestellungen

Was sind die Hauptmerkmale von JSON?

- Einfach mit zu arbeiten vor allem im web bereich mit JS und Java backends
- kleiner als xml

Was sind die Hauptmerkmale von XML?

- Überall irgendwie unterstützt
- einfach Selber zu schreiben/lesen zum testen

Wie unterscheidet die Applikation in diesem Beispiel, ob die Rückgabewerte in JSON oder XML zurückgegeben werden?

Mithilfe des `Accept: application/xml` im curl beispielsweise. Damit weiß die REST API über welche schnittstelle sie die Daten formatieren und zurückgeben muss.

Was zeichnet ein REST Interface aus und warum wird es heute sehr oft als standardisierte Schnittstelle verwendet?

Ein Interface ist ein REST interface wenn es die Begrenzungen einhält die für ein REST interface vorgesehen sind. Am anfang des WEBs gab es noch nicht wirklich Vorgaben für wie APIs und schnittstellen auszusehen haben also haben sich leute gedacht, 'lass das doch mal standartisieren' seither verwenden die meisten internetseiten REST damit Devs es einfach haben auf die daten zuzugreifen.

Was ist der Unterschied zwischen den HTTP Methoden GET, POST, PUT und PATCH?

Methode	Funktion
GET	Daten Abfragen
PUT	Ersetzt einen kompletten Datensatz mit einem anderen vollständigen Datensatz

Methode	Funktion
POST	Erstellt neue Daten
PATCH	Erstetzt einzelne Felder aus einem Datensatz

Ich will eine neue Request Methode / Pfad in meinem REST Interface zur Verfügung stellen, welche Stellen des Quellcodes muss ich verändern?

Ich muss im controller eine neue `@RequestMapping` oder `@GetMapping` machen und einen Pfad zuweisen, in der Methode darunter muss ich definieren welche daten zurück gegeben werden. Als return type muss ich entweder `Object` oder den Datentyp selber zurückgeben damit Spring mir das automatisch zu JSON oder XML wrapen kann.

Kann man OpenAPI und Swagger vergleichen? Wenn ja, dann vergleichen Sie beide Themengebiete. Wenn nein, dann begründe die Antwort.

Nein kann man nicht, Da das Swagger UI nur eine andere Visualisierung der gleichen OpenAPI dokumentation ist. Genauso wie man VW nicht mit Seat vergleichen kann weil die beiden bis auf das aussehen das gleiche sind

Wo liegt im folgenden Code-Snippet der Fehler? Begründe die Antwort.

```
@GetMapping("/stations/add")
public String createStation(@RequestParam String name, @RequestParam double
warning) {
    stationService.save(new Station(name, warning));
    return "Station successfully created!";
}
```

In der Dependancy Injection.

Man muss ein `@PostMapping` machen damit der nutzer Daten hochladen kann. Zurzeit wird der nutzer nur den String zurück bekommen wenn überhaupt. wahrscheinlich wird das programm in der zeile darüber schon abstürzen.

WEB-UI

> Generate me a simple WEB-ui that acceses the REST API and visualizes the recived data on a simple homepage. make it auto update every x seconds (changeable parameter) and

> make it interactive to be able to click each station to see its data. make it reachable under localhost:8080/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="theme-color" content="#f4f7f8">
    <title>Flood Monitor</title>
    <style>
        :root {
            color-scheme: light;
            font-family: Inter, ui-sans-serif, system-ui, -apple-system,
BlinkMacSystemFont, "Segoe UI", sans-serif;
            color: #15303a;
            background: #f4f7f8;
            font-synthesis: none;
            text-rendering: optimizeLegibility;
        }

        * { box-sizing: border-box; }
        body { margin: 0; min-width: 320px; }
        button, input { font: inherit; }
        button { cursor: pointer; }

        .page { max-width: 1180px; margin: 0 auto; padding: 42px 28px 64px; }
        .topbar { display: flex; align-items: center; justify-content: space-
between; gap: 24px; margin-bottom: 44px; }
        .brand { display: flex; align-items: center; gap: 12px; font-weight:
750; letter-spacing: -.03em; }
        .brand-mark { display: grid; width: 38px; height: 38px; place-items:
center; border-radius: 12px; color: white; background: #087f8c; }
        .brand-mark svg { width: 22px; height: 22px; }
        .connection { display: flex; align-items: center; gap: 8px; color:
#61747b; font-size: 13px; }
        .dot { width: 8px; height: 8px; border-radius: 50%; background:
#26a269; }
        .dot.error { background: #d44e4e; }

        .intro { display: flex; align-items: flex-end; justify-content: space-
between; gap: 28px; margin-bottom: 28px; }
        .eyebrow { margin: 0 0 10px; color: #087f8c; font-size: 12px; font-
weight: 750; letter-spacing: .13em; text-transform: uppercase; }
```

```
        h1 { margin: 0; font-size: clamp(30px, 5vw, 42px); line-height: 1.1;
letter-spacing: -.045em; }
        .subtitle { margin: 12px 0 0; color: #718188; font-size: 15px; }
        .refresh-controls { display: flex; align-items: center; gap: 10px;
flex-wrap: wrap; justify-content: flex-end; }
        .interval-control { display: flex; align-items: center; gap: 8px;
color: #61747b; font-size: 13px; }
        .interval-control input { width: 72px; padding: 10px 9px; border: 1px
solid #dce5e7; border-radius: 9px; color: #15303a; background: white; }
        .button { min-height: 40px; padding: 0 15px; border: 0; border-radius:
9px; color: white; background: #087f8c; font-size: 13px; font-weight: 700;
transition: background .15s ease; }
        .button:hover { background: #066b75; }
        .button:disabled { cursor: wait; opacity: .65; }

        .notice { display: none; margin: 0 0 20px; padding: 12px 15px; border:
1px solid #f1cdcd; border-radius: 10px; color: #913c3c; background: #fff5f5;
font-size: 13px; }
        .notice.visible { display: block; }
        .summary { display: grid; grid-template-columns: repeat(3, minmax(0,
1fr)); gap: 14px; margin-bottom: 34px; }
        .summary-card, .station-card, .detail-panel { border: 1px solid
#e5ebed; border-radius: 14px; background: white; box-shadow: 0 3px 12px rgb(20
48 58 / 3%); }
        .summary-card { padding: 19px 21px; }
        .summary-label { color: #798990; font-size: 12px; font-weight: 650; }
        .summary-value { margin-top: 9px; font-size: 25px; font-weight: 750;
letter-spacing: -.04em; }
        .summary-value small { color: #728187; font-size: 13px; font-weight:
550; letter-spacing: 0; }

        .section-heading { display: flex; align-items: baseline; justify-
content: space-between; gap: 16px; margin-bottom: 14px; }
        h2 { margin: 0; font-size: 18px; letter-spacing: -.025em; }
        .updated { color: #849399; font-size: 12px; }
        .station-grid { display: grid; grid-template-columns: repeat(auto-fit,
minmax(215px, 1fr)); gap: 14px; }
        .station-card { display: block; width: 100%; padding: 19px; text-align:
left; color: inherit; transition: border-color .15s ease, transform .15s ease,
box-shadow .15s ease; }
        .station-card:hover { transform: translateY(-2px); border-color:
#9acbd0; box-shadow: 0 8px 20px rgb(20 48 58 / 7%); }
        .station-card.selected { border-color: #087f8c; box-shadow: 0 0 0 2px
rgb(8 127 140 / 12%); }
        .station-card-top { display: flex; align-items: center; justify-
```

```
content: space-between; gap: 12px; }
        .station-name { overflow: hidden; font-size: 14px; font-weight: 700;
text-overflow: ellipsis; white-space: nowrap; }
        .station-id { flex: none; color: #91a0a5; font-size: 11px; }
        .level-label { margin-top: 24px; color: #849399; font-size: 11px; }
        .level-value { margin-top: 3px; color: #087f8c; font-size: 29px; font-
weight: 750; letter-spacing: -.05em; }
        .level-value small { font-size: 14px; font-weight: 600; letter-spacing:
0; }
        .card-footer { display: flex; justify-content: space-between; margin-
top: 18px; padding-top: 13px; border-top: 1px solid #edf1f2; color: #849399;
font-size: 11px; }
        .empty { grid-column: 1 / -1; padding: 32px 20px; border: 1px dashed
#d7e1e3; border-radius: 14px; color: #718188; text-align: center; }

        .detail-panel { display: none; margin-top: 24px; padding: 22px; }
        .detail-panel.visible { display: block; }
        .detail-heading { display: flex; align-items: flex-start; justify-
content: space-between; gap: 16px; margin-bottom: 20px; }
        .detail-heading h2 { font-size: 20px; }
        .detail-subtitle { margin: 6px 0 0; color: #849399; font-size: 12px; }
        .close-button { width: 32px; height: 32px; border: 0; border-radius:
8px; color: #718188; background: #f2f6f6; font-size: 19px; line-height: 1; }
        .detail-grid { display: grid; grid-template-columns: repeat(4,
minmax(0, 1fr)); gap: 12px; }
        .detail-item { padding: 15px; border-radius: 10px; background: #f6f9f9;
}
        .detail-item span { display: block; color: #849399; font-size: 11px; }
        .detail-item strong { display: block; margin-top: 8px; font-size: 17px;
font-weight: 700; }
        .footer { margin-top: 30px; color: #9aa7ab; font-size: 11px; text-
align: center; }

        @media (max-width: 680px) {
            .page { padding: 26px 16px 42px; }
            .topbar { margin-bottom: 34px; }
            .intro { align-items: flex-start; flex-direction: column; }
            .refresh-controls { justify-content: flex-start; }
            .summary { grid-template-columns: 1fr; gap: 9px; }
            .summary-card { padding: 15px 17px; }
            .summary-value { font-size: 22px; }
            .detail-grid { grid-template-columns: repeat(2, minmax(0, 1fr)); }
        }
    </style>
</head>
```

```
<body>
<main class="page">
    <header class="topbar">
        <div class="brand">
            <span class="brand-mark" aria-hidden="true">
                <svg viewBox="0 0 24 24" fill="none">
                    <path d="M12 3C9.3 7.1 5 11.1 5 15a7 7 0 0 0 14 0c0-3.9-
4.3-7.9-7-12Z" stroke="currentColor" stroke-width="2" stroke-linecap="round"
stroke-linejoin="round"/>
                    <path d="M8.5 15.5c.3 1.6 1.5 2.5 3.2 2.7"
stroke="currentColor" stroke-width="2" stroke-linecap="round"/>
                </svg>
            </span>
            <span>Flood Monitor</span>
        </div>
        <div class="connection" aria-live="polite">
            <span class="dot" id="connection-dot"></span>
            <span id="connection-label">Connecting to API</span>
        </div>
    </header>

    <section class="intro">
        <div>
            <p class="eyebrow">Live overview</p>
            <h1>Water stations</h1>
            <p class="subtitle">Current readings from your flood monitoring
network.</p>
        </div>
        <div class="refresh-controls">
            <label class="interval-control" for="refresh-interval">
                Update every
                <input id="refresh-interval" type="number" min="1" max="3600"
step="1" value="10" inputmode="numeric" aria-label="Refresh interval in
seconds">
                seconds
            </label>
            <button class="button" id="refresh-button" type="button">Refresh
now</button>
        </div>
    </section>

    <div class="notice" id="error-notice" role="alert"></div>

    <section class="summary" aria-label="Station summary">
        <article class="summary-card">
```

```
            <div class="summary-label">Stations online</div>
            <div class="summary-value" id="station-count">–</div>
        </article>
        <article class="summary-card">
            <div class="summary-label">Highest water level</div>
            <div class="summary-value" id="highest-level">–</div>
        </article>
        <article class="summary-card">
            <div class="summary-label">Next update</div>
            <div class="summary-value"><span id="countdown">–</span> <small
id="countdown-unit"></small></div>
        </article>
    </section>

    <section aria-labelledby="stations-heading">
        <div class="section-heading">
            <h2 id="stations-heading">All stations</h2>
            <span class="updated" id="last-updated">Waiting for first
update</span>
        </div>
        <div class="station-grid" id="station-grid" aria-live="polite">
            <div class="empty">Loading station data…</div>
        </div>
    </section>

    <section class="detail-panel" id="detail-panel" aria-live="polite" aria-
label="Selected station details">
        <div class="detail-heading">
            <div>
                <h2 id="detail-name"></h2>
                <p class="detail-subtitle" id="detail-id"></p>
            </div>
            <button class="close-button" id="close-details" type="button" aria-
label="Close station details">×</button>
        </div>
        <div class="detail-grid" id="detail-grid"></div>
    </section>

    <footer class="footer">Data provided by the Flood Monitor REST API</footer>
</main>

<script>
    const API_URL = "/api/v1/stations";
    const grid = document.getElementById("station-grid");
    const notice = document.getElementById("error-notice");
```

```
    const intervalInput = document.getElementById("refresh-interval");
    const refreshButton = document.getElementById("refresh-button");
    const detailPanel = document.getElementById("detail-panel");
    const stationCards = new Map();
    const detailValues = new Map();
    let stations = [];
    let selectedStationId = null;
    let refreshTimer = null;
    let countdownTimer = null;
    let nextRefreshAt = 0;
    let isRefreshing = false;

    for (const [key, label] of [
        ["level", "Water level"],
        ["temperature", "Water temperature"],
        ["flow", "Water flow"],
        ["timestamp", "Last reading"]
    ]) {
        const item = document.createElement("div");
        item.className = "detail-item";
        const name = document.createElement("span");
        name.textContent = label;
        const value = document.createElement("strong");
        item.append(name, value);
        document.getElementById("detail-grid").append(item);
        detailValues.set(key, value);
    }

    function safeNumber(value) {
        const number = Number(value);
        return Number.isFinite(number) ? number : null;
    }

    function formatNumber(value, maximumFractionDigits = 1) {
        const number = safeNumber(value);
        return number === null ? "–" : new Intl.NumberFormat(undefined, {
maximumFractionDigits }).format(number);
    }

    function formatTimestamp(value) {
        if (!value) return "Unknown";
        const date = new Date(value);
        return Number.isNaN(date.getTime()) ? String(value) :
date.toLocaleString();
    }
```

```
    function showError(message) {
        notice.textContent = message;
        notice.classList.add("visible");
        document.getElementById("connection-dot").classList.add("error");
        document.getElementById("connection-label").textContent = "API
unavailable";
    }

    function clearError() {
        notice.textContent = "";
        notice.classList.remove("visible");
        document.getElementById("connection-dot").classList.remove("error");
        document.getElementById("connection-label").textContent = "Connected to
API";
    }

    function renderSummary() {
        document.getElementById("station-count").textContent =
String(stations.length);
        const highest = stations.reduce((current, station) => {
            const level = safeNumber(station.waterLevel);
            return level !== null && (current === null || level > current) ?
level : current;
        }, null);
        document.getElementById("highest-level").textContent =
            highest === null ? "–" : `${formatNumber(highest, 2)} m`;
    }

    function createStationCard(id) {
        const card = document.createElement("button");
        card.type = "button";
        card.className = "station-card";
        card.setAttribute("aria-pressed", "false");
        card.addEventListener("click", () => selectStation(id));

        const top = document.createElement("div");
        top.className = "station-card-top";
        const name = document.createElement("span");
        name.className = "station-name";
        const stationId = document.createElement("span");
        stationId.className = "station-id";
        top.append(name, stationId);

        const levelLabel = document.createElement("div");
```

```
        levelLabel.className = "level-label";
        levelLabel.textContent = "Water level";
        const level = document.createElement("div");
        level.className = "level-value";
        const levelNumber = document.createElement("span");
        const unit = document.createElement("small");
        level.append(levelNumber, unit);

        const footer = document.createElement("div");
        footer.className = "card-footer";
        const temperature = document.createElement("span");
        const flow = document.createElement("span");
        footer.append(temperature, flow);
        card.append(top, levelLabel, level, footer);

        return { element: card, name, id: stationId, levelNumber, unit,
temperature, flow };
    }

    function updateStationCard(card, station) {
        const id = String(station.id);
        const isSelected = id === selectedStationId;
        card.name.textContent = station.stationName || `Station ${id}`;
        card.id.textContent = `#${id}`;
        card.levelNumber.textContent = formatNumber(station.waterLevel, 2);
        card.unit.textContent = ` ${station.unit || "m"}`;
        card.temperature.textContent =
`${formatNumber(station.waterTemperature)} °C`;
        card.flow.textContent = `Flow ${formatNumber(station.waterflow)}`;
        card.element.classList.toggle("selected", isSelected);
        card.element.setAttribute("aria-pressed", String(isSelected));
    }

    function renderStations() {
        const stationIds = new Set(stations.map(station =>
String(station.id)));
        for (const [id, card] of stationCards) {
            if (!stationIds.has(id)) {
                card.element.remove();
                stationCards.delete(id);
            }
        }

        if (stations.length === 0) {
            const empty = document.createElement("div");
```

```
            empty.className = "empty";
            empty.textContent = "No stations are currently available.";
            grid.replaceChildren(empty);
            return;
        }

        grid.querySelector(".empty")?.remove();
        stations.forEach((station, index) => {
            const id = String(station.id);
            let card = stationCards.get(id);
            if (!card) {
                card = createStationCard(id);
                stationCards.set(id, card);
            }
            updateStationCard(card, station);
            const current = grid.children[index];
            if (current !== card.element) {
                grid.insertBefore(card.element, current || null);
            }
        });
    }

    function renderDetails(station) {
        document.getElementById("detail-name").textContent =
station.stationName || `Station ${station.id}`;
        document.getElementById("detail-id").textContent = `Station ID:
${station.id}`;
        detailValues.get("level").textContent =
`${formatNumber(station.waterLevel, 2)} ${station.unit || "m"}`;
        detailValues.get("temperature").textContent =
`${formatNumber(station.waterTemperature)} °C`;
        detailValues.get("flow").textContent = formatNumber(station.waterflow);
        detailValues.get("timestamp").textContent =
formatTimestamp(station.timestamp);
        detailPanel.classList.add("visible");
    }

    async function selectStation(id) {
        selectedStationId = id;
        renderStations();
        detailPanel.classList.add("visible");
        document.getElementById("detail-name").textContent = "Loading
station…";
        document.getElementById("detail-id").textContent = `Station ID: ${id}`;
        for (const value of detailValues.values()) value.textContent = "—";
```

```
        try {
            const response = await
fetch(`${API_URL}/${encodeURIComponent(id)}`, { headers: { Accept:
"application/json" } });
            if (!response.ok) throw new Error(`Could not load station ${id}
(HTTP ${response.status}).`);
            const station = await response.json();
            if (selectedStationId === id) renderDetails(station);
        } catch (error) {
            if (selectedStationId === id) showError(error.message || "Could not
load the selected station.");
        }
    }

    async function loadStations() {
        if (isRefreshing) return;
        isRefreshing = true;
        refreshButton.disabled = true;
        try {
            const response = await fetch(API_URL, { headers: { Accept:
"application/json" }, cache: "no-store" });
            if (!response.ok) throw new Error(`Could not load stations (HTTP
${response.status}).`);
            const data = await response.json();
            if (!Array.isArray(data)) throw new Error("The stations API
returned an unexpected response.");
            stations = data;
            clearError();
            renderSummary();
            renderStations();
            document.getElementById("last-updated").textContent = `Updated
${new Date().toLocaleTimeString()}`;
            if (selectedStationId !== null) {
                const selectedStation = stations.find(station =>
String(station.id) === selectedStationId);
                if (selectedStation) {
                    renderDetails(selectedStation);
                } else {
                    selectedStationId = null;
                    detailPanel.classList.remove("visible");
                }
            }
        } catch (error) {
            showError(error.message || "Could not load station data. Check that
the Flood Monitor server is running.");
```

```
            if (stations.length === 0) {
                grid.replaceChildren();
                const empty = document.createElement("div");
                empty.className = "empty";
                empty.textContent = "Station data could not be loaded.";
                grid.append(empty);
            }
        } finally {
            isRefreshing = false;
            refreshButton.disabled = false;
            scheduleRefresh();
        }
    }

    function getIntervalSeconds() {
        const parsed = Number.parseInt(intervalInput.value, 10);
        const seconds = Number.isFinite(parsed) ? Math.min(3600, Math.max(1,
parsed)) : 10;
        intervalInput.value = String(seconds);
        return seconds;
    }

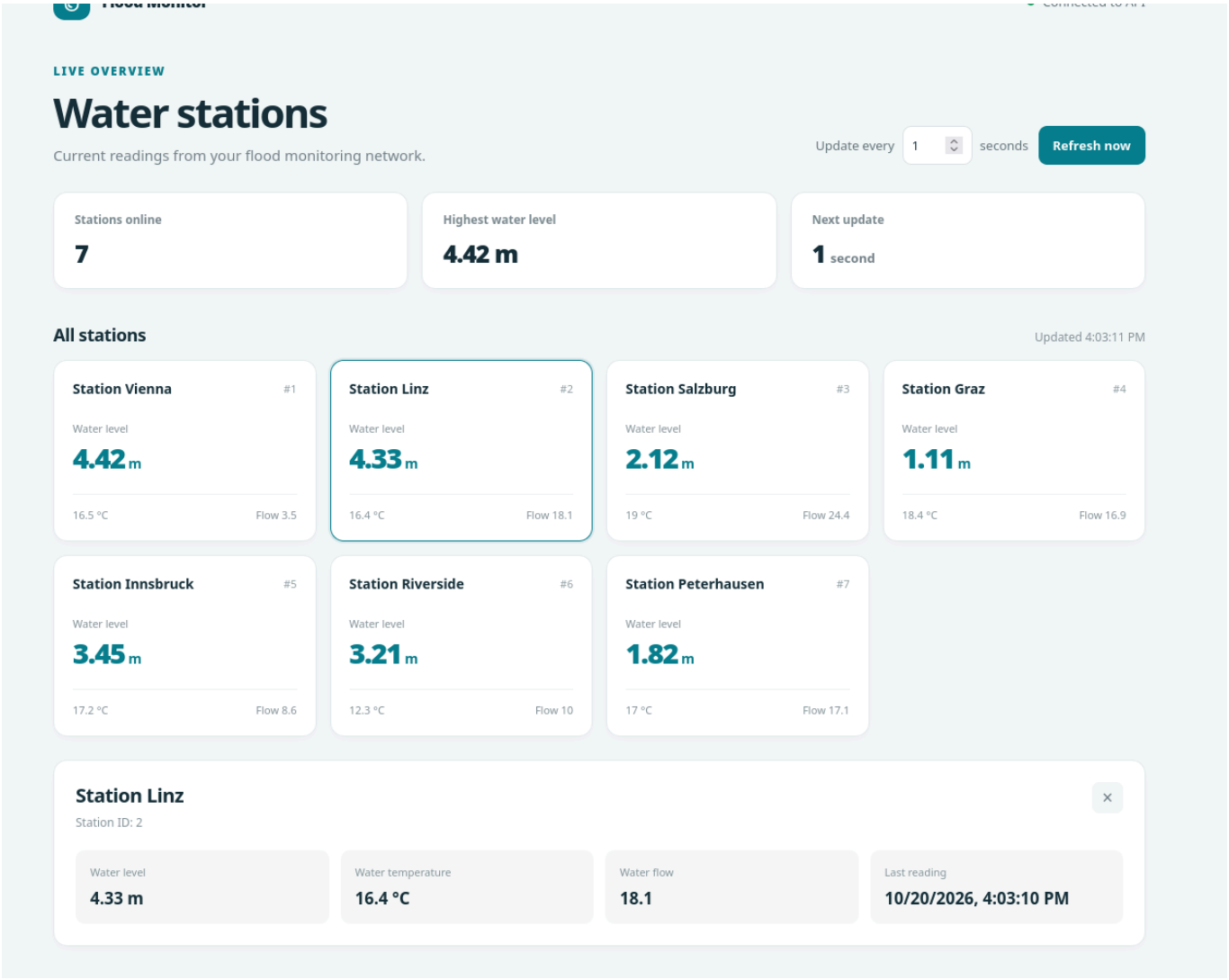
    function scheduleRefresh() {
        window.clearTimeout(refreshTimer);
        nextRefreshAt = Date.now() + getIntervalSeconds() * 1000;
        refreshTimer = window.setTimeout(loadStations, getIntervalSeconds() *
1000);
        updateCountdown();
    }

    function updateCountdown() {
        const remaining = Math.max(0, Math.ceil((nextRefreshAt - Date.now()) /
1000));
        document.getElementById("countdown").textContent = String(remaining);
        document.getElementById("countdown-unit").textContent = remaining === 1
? "second" : "seconds";
    }

    refreshButton.addEventListener("click", () => {
        window.clearTimeout(refreshTimer);
        loadStations();
    });
    intervalInput.addEventListener("change", scheduleRefresh);
    document.getElementById("close-details").addEventListener("click", () => {
        selectedStationId = null;
```

```
        detailPanel.classList.remove("visible");
        renderStations();
    });

    window.setInterval(updateCountdown, 1000);
    loadStations();
</script>
</body>
</html>
```



Error handling

Error handling seiten in Spring sind sehr einfach.

um eine generic error seite bei irgend einem fehler zu machen muss man unter `/recources/templates/` eine `error.html` erstellen. Wenn ein fehler auftritt wird diese dann

aufgerufen.

Bei spezifisch häufigen fehlern wie 404 oder 403 kann man auch eine eigene seite nur für den fehler oder die ganze 400 gruppe machen.

Dafür muss man eine `4xx.html` oder `404.html` im folder `/recources/public/error/` erstellen und darin dann die webseite machen.

Sources & Prompts:

> Generate me a simple WEB-ui that acceses the REST API and visualizes the recived data on a simple homepage. make it auto update every x seconds (changeable parameter) and make it interactive to be able to click each station to see its data. make it reachable under localhost:8080/index.html

> https://en.wikipedia.org/wiki/REST

> https://devwithtools.org/blog/http-methods-explained

> https://springdoc.org